

04 — Guide de modification en direct (anti-panique) — DÉTAILLÉ

Schéma mental du projet (à réciter) : Frontend/*.html (structure) + Frontend/script/*.js (fetch \${API_BASE_URL}/route) → **Nginx** /api → API/route/<domaine>.go (déclare la route) → API/admin/<domaine>.go (handler) → API/bdd/<domaine>Req.go (SQL) → API/models/<domaine>.go (struct).

Réflexe après CHAQUE modif : - Modif **backend (Go)** → docker compose up -d --build backend (≈20-40 s) puis tester. - Modif **frontend (HTML/JS/CSS)** → WEB_PORT=8088 docker compose up -d --build frontend **OU** ouvrir via WAMP + **Ctrl+Shift+R**. - Toujours vérifier dans **F12** → **Réseau** le code HTTP de l'appel.

0. Les 3 gestes qui sauvent

```
# Compiler l'API sans démarrer (détecte les erreurs Go en 2 s)
cd API && go build ./...

# Reconstruire + relancer un seul service
docker compose up -d --build backend      # ou frontend

# Voir l'erreur exacte
docker compose logs -f backend
```

Récupérer un token pour tester en curl (à garder sous la main) :

```
TOKEN=$(curl -s -X POST http://localhost:8081/admin/login \
-H "Content-Type: application/json" \
-d '{"email":"test.admin@renova.test","mot_de_passe":"TON_MDP"}' | grep -o '"token":"[^"]*"' | cut -d'"' -f4)
echo $TOKEN
```

A. Ajouter un champ dans un formulaire

Exemple complet : ajouter `telephone` à l'utilisateur

1. Base de données — ajouter la colonne :

```
docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026
```

```
ALTER TABLE utilisateur ADD COLUMN telephone VARCHAR(20) NULL;
```

2. Struct Go — API/models/users.go : ajouter dans le struct `User` :

```
Telephone string `json:"telephone"`
```

3. Requête SQL — API/bdd/userReq.go : ajouter `telephone` dans le `SELECT` **et** dans le `Scan` correspondant.

```
// AVANT
"SELECT id, nom, prenom, email, role, score FROM utilisateur WHERE id = ?"
// APRÈS
```

```
"SELECT id, nom, prenom, email, role, score, COALESCE(telephone, '') FROM utilisateur WHERE id = ?"
// et dans .Scan(...) ajouter &user.Telephone à la fin, dans le MÊME ordre
```

⚠ Règle d'or : **l'ordre des colonnes du SELECT = l'ordre des &champ dans le Scan** .

4. Handler — `API/admin/users.go` : lire le champ à la création/modif :

```
user.Telephone = r.FormValue("telephone") // si formulaire multipart
// ou déjà rempli par json.NewDecoder(r.Body).Decode(&user) si JSON
```

5. Frontend — HTML (formulaire) :

```
<input type="text" id="add-telephone" placeholder="Téléphone">
```

JS (`Frontend/script/admin/user.js` , fonction `CreateUser`) :

```
body: JSON.stringify({ nom, prenom, email, role, mot_de_passe: mdp, telephone: document.getElementById("add-telephone").value })
```

6. Rebuild + test :

```
docker compose up -d --build backend
curl -X PUT http://localhost:8081/admin/users/modify/1 -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"telephone":"0600000000"}'
# puis vérifier en base :
docker exec uc_mysql mysql -uupcycle -pupcyclePass123 pa2026 -e "SELECT telephone FROM utilisateur WHERE id=1;"
```

Variante rapide (front seulement) : ajouter un champ à une annonce

- Le formulaire d'annonce envoie un `FormData` (multipart). Ajouter :

```
formData.append("ville", document.getElementById("ville").value);
```

- Côté Go (`API/admin/annonce.go`), le lire : `ann.Ville = r.FormValue("ville")` (le champ existe déjà ici : `ville`).

B. Ajouter un filtre

Option 1 (RECO en direct) — filtre 100 % front, aucun rebuild

Sur la page annonces, les données sont déjà en mémoire dans `allAnnonces` (`Frontend/script/annonces/annonceAll.js`). Modèle existant = `toggleFilter` / `buildCategoryFilter` .

Exemple : filtrer par **ville** :

```
function filtrerParVille(ville) {
  const filtered = allAnnonces.filter(
    (a) => (a.ville || "").toLowerCase() === ville.toLowerCase()
  );
  displayAnnonces(filtered);
}
```

Brancher sur un bouton : `<button onclick="filtrerParVille('Paris')">Paris</button>` . **Test** : recharger `/annonces` , cliquer → seules les annonces de Paris s'affichent.

Option 2 — filtre côté serveur

API — `API/admin/annonce.go` : `ville := r.URL.Query().Get("ville")` . **DB** — `API/bdd/annonceReq.go` : ajouter la condition (garder le paramètre ?) :

```
"SELECT ... FROM annonce WHERE UPPER(titre) LIKE ? AND (? = '' OR ville = ?)"
// args : search, ville, ville
```

Test : `curl "http://localhost:8081/admin/annonces/search?ville=Paris" -H "Authorization: Bearer $TOKEN"` .

C. Modifier une règle métier

C1. Taux de commission (5 % → 10 %)

- **Fichier** : `API/admin/stripe.go` .
- Chercher (Ctrl+F) : `* 5` / `100` et `0.05` .

```
// AVANT
commission := (unitAmount * 5) / 100
commissionEuros := prix * 0.05
// APRÈS (10 %)
commission := (unitAmount * 10) / 100
commissionEuros := prix * 0.10
```

- **Rebuild + test** : `docker compose up -d --build backend` , puis un achat → `GET /admin/finance/overview` doit refléter la nouvelle commission.

C2. Upcycling Score (coefficients)

- **Fichier** : `API/bdd/boxReq.go` , fonction `CalculateAndAddScore` .

```
coefficients := map[string]float64{
    "textile": 15.0, "metal": 10.0, "bois": 5.0, "plastique": 8.0, "autre": 3.0,
}
gainScore := poids * coef
```

- Exemple : doubler la valeur du textile → `"textile": 30.0` .
- **Test** : `GET /api/user/stats?user_id=1` avant/après un dépôt validé.
- **À savoir (honnête)** : actuellement `materiau` n'est pas relu depuis la base → `coef` vaut toujours `autre` (3.0). Pour corriger : ajouter `materiau` (via la catégorie) dans le `SELECT` de la fonction.

C3. Seuil / délai (ex : nettoyage des box réservées)

- **Fichier** : `API/bdd/boxReq.go` , `GarbageCollectBox` : `INTERVAL 2 DAY` . Changer la valeur pour ajuster le délai.

D. Protéger une route par rôle

- **Middlewares** : `API/auth/jwt.go`
- `VerifyTokenMiddleware(next)` → exige un JWT valide (header `Authorization: Bearer ...`), met `userID` / `userRole` dans le `context` .
- `VerifyRoleMiddleware(next, roles...)` → 403 si le rôle n'est pas dans la liste.
- **Application** dans le fichier de route, ex. `API/route/users.go` :

```
// Accessible Salarié ET Admin :
http.HandleFunc("GET /admin/users",
```

```
auth.VerifyRoleMiddleware(admin.GetAllUsers, "Salarié", "Administrateur"))
// Réserver aux Admins uniquement :
http.HandleFunc("GET /admin/users",
    auth.VerifyRoleMiddleware(admin.GetAllUsers, "Administrateur"))
```

- **Test à deux comptes :**

```
# Token admin -> 200
curl -i http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN_ADMIN"
# Token particulier -> 403 "Accès refusé"
curl -i http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN_USER"
# Sans token -> 401
curl -i http://localhost:8081/admin/users
```

E. Ajouter une colonne dans un tableau admin

Exemple : afficher l'email dans la liste des utilisateurs

Front — Frontend/script/admin/user.js, fonction `afficherPageUsers` : 1. Dans `headerHTML` (bloc `u-thead`), ajouter une cellule d'en-tête :

```
<div>Email</div>
```

1. Dans la boucle `usersPage.forEach` (bloc `u-row`), ajouter la cellule :

```
<div class="u-email">${user.email}</div>
```

1. Si le tableau est en CSS grid, ajuster le nombre de colonnes dans `Frontend/style/admin.css` (`.u-thead`, `.u-row` → `grid-template-columns`).

Test : recharger `/admin/users`, la colonne apparaît. (Aucun rebuild backend si le champ est déjà renvoyé par l'API.)

F. Ajouter un endpoint simple

Exemple complet : `GET /admin/users/count`

1. **Route** — `API/route/users.go` :

```
http.HandleFunc("OPTIONS /admin/users/count", admin.CountUsers)
http.HandleFunc("GET /admin/users/count", auth.VerifyTokenMiddleware(admin.CountUsers))
```

2. **Handler** — `API/admin/users.go` :

```
func CountUsers(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Access-Control-Allow-Origin", "*")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
    if r.Method == "OPTIONS" { w.WriteHeader(http.StatusOK); return }

    n, err := bdd.CountUsers()
    if err != nil {
        http.Error(w, "erreur count", http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "application/json")
```

```
json.NewEncoder(w).Encode(map[string]int{"count": n})
}
```

3. Requête DB — API/bdd/userReq.go :

```
func CountUsers() (int, error) {
    var n int
    err := Db.QueryRow("SELECT COUNT(*) FROM utilisateur").Scan(&n)
    return n, err
}
```

4. Rebuild + test :

```
docker compose up -d --build backend
curl http://localhost:8081/admin/users/count -H "Authorization: Bearer $TOKEN"
# -> {"count": 16}
```

⚠ **Toujours** ajouter la route `OPTIONS` + les 2 headers CORS, sinon erreur CORS en dev local.

Exemple 2 : GET /admin/stats (mini dashboard)

```
// route
http.HandleFunc("OPTIONS /admin/stats", admin.GetStats)
http.HandleFunc("GET /admin/stats", auth.VerifyRoleMiddleware(admin.GetStats, "Administrateur"))
// handler (API/admin/users.go ou un nouveau fichier)
func GetStats(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Access-Control-Allow-Origin", "")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
    if r.Method == "OPTIONS" { w.WriteHeader(http.StatusOK); return }
    var users, annonces int
    bdd.Db.QueryRow("SELECT COUNT(*) FROM utilisateur").Scan(&users)
    bdd.Db.QueryRow("SELECT COUNT(*) FROM annonce").Scan(&annonces)
    json.NewEncoder(w).Encode(map[string]int{"users": users, "annonces": annonces})
}
```

Exemple 3 : PATCH / PUT d'un statut — PUT /admin/annonces/validate/{id} existe déjà

Modèle de handler qui change un statut (déjà dans API/admin/annonce.go) :

```
id, _ := strconv.Atoi(r.PathValue("id"))
bdd.Db.Exec("UPDATE annonce SET statut_validation = 'Validé' WHERE id = ?", id)
```

G. Ajouter une entrée de menu / un lien de navigation

- **HTML** de la page → ajouter dans la `<nav>` : `<a class="nav-link" href="/annonces">Annonces</a>`.
- Utiliser les **URLs propres** (`/login`, `/admin`, `/annonces` ...) — elles sont réécrites par `Frontend/nginx.conf`.
- Le surlignage du lien actif (admin) est géré par `Frontend/script/auth_guard.js` (surlignerLienActif).

H. Ajouter une traduction (texte multilingue)

1. **HTML** : mettre `data-i18n="ma.cle"` sur l'élément (le texte par défaut reste le fallback).
2. **Base** : ajouter la clé dans les 2 langues :

```
INSERT INTO translations (lang_code, msg_key, msg_value) VALUES
('fr','ma.cle','Mon texte'), ('en','ma.cle','My text');
```

1. `translate.js` remplace automatiquement au chargement. **Test** : changer de langue via le sélecteur en bas à droite.

J. La pagination (question probable : « comment paginez-vous ? »)

Il y a **2 façons** de paginer. Sache expliquer les deux et **pourquoi j'ai choisi la 1ère**.

J.1 — Ce qui est fait dans le projet : pagination CÔTÉ CLIENT (JS)

- **Fichier** : `Frontend/script/admin/user.js`.
- **Principe** : l'API renvoie **tous** les utilisateurs (`GET /admin/users`), on les garde en mémoire, et on affiche seulement une **tranche de 10** avec `slice`. Les boutons changent l'index de page et ré-affichent.
- **Pourquoi ce choix ?** Le nombre d'utilisateurs est petit → simple, aucune requête réseau à chaque changement de page, tri/recherche instantanés côté client.

```
let tousLesUsers = [];           // toutes les données chargées une fois
let pageUsers = 1;               // page courante
const USERS_PAR_PAGE = 10;      // taille d'une page

function AfficherTableau(users) { // reçoit la liste complète de l'API
  tousLesUsers = users || [];
  pageUsers = 1;
  afficherPageUsers();
}

function changerPageUsers(delta) { // bouton Précédent (-1) / Suivant (+1)
  pageUsers += delta;
  if (pageUsers < 1) pageUsers = 1;
  afficherPageUsers();
}

function afficherPageUsers() {
  const nbPages = Math.max(1, Math.ceil(tousLesUsers.length / USERS_PAR_PAGE));
  if (pageUsers > nbPages) pageUsers = nbPages;
  const debut = (pageUsers - 1) * USERS_PAR_PAGE; // index de départ
  const usersPage = tousLesUsers.slice(debut, debut + USERS_PAR_PAGE); // la tranche
  // ... on génère le HTML uniquement pour usersPage ...
  // + une barre "Page X / Y" avec 2 boutons onclick="changerPageUsers(-1|1)"
}
```

Points clés à dire : `Math.ceil(total / taille)` = nombre de pages ; `slice(debut, debut + taille)` = la tranche ; on désactive les boutons aux extrémités.

J.2 — Comment on le ferait CÔTÉ SERVEUR en Go (SQL LIMIT / OFFSET)

À utiliser si la table devient **grosse** (des milliers de lignes) : on ne renvoie qu'une page depuis la base.

Requête SQL — la clé, c'est `LIMIT taille OFFSET (page-1)*taille` :

```
SELECT id, nom, prenom, email, role FROM utilisateur
ORDER BY id
LIMIT ? OFFSET ?;      -- LIMIT = 10, OFFSET = (page-1)*10
```

DB — `API/bdd/userReq.go` :

```
func GetUsersPagines(page, taille int) ([]models.User, int, error) {
    if page < 1 { page = 1 }
    offset := (page - 1) * taille

    // total (pour calculer le nombre de pages)
    var total int
    Db.QueryRow("SELECT COUNT(*) FROM utilisateur").Scan(&total)

    rows, err := Db.Query(
        "SELECT id, nom, prenom, email, role FROM utilisateur ORDER BY id LIMIT ? OFFSET ?",
        taille, offset,
    )
    if err != nil { return nil, 0, err }
    defer rows.Close()

    var users []models.User
    for rows.Next() {
        var u models.User
        rows.Scan(&u.Id, &u.Nom, &u.Prenom, &u.Email, &u.Role)
        users = append(users, u)
    }
    return users, total, nil
}
```

Handler — `API/admin/users.go` (lit `?page=` et `?limit=`):

```
func GetUsersPagines(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Access-Control-Allow-Origin", "")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
    if r.Method == "OPTIONS" { w.WriteHeader(http.StatusOK); return }

    page, _ := strconv.Atoi(r.URL.Query().Get("page"))
    taille, _ := strconv.Atoi(r.URL.Query().Get("limit"))
    if taille <= 0 { taille = 10 }

    users, total, err := bdd.GetUsersPagines(page, taille)
    if err != nil { http.Error(w, "erreur", 500); return }

    json.NewEncoder(w).Encode(map[string]interface{}{
        "users": users,
        "total": total,
        "page": page,
        "pages": (total + taille - 1) / taille, // arrondi supérieur
    })
}
```

Route — `API/route/users.go` :

```
http.HandleFunc("OPTIONS /admin/users/paginated", admin.GetUsersPagines)
http.HandleFunc("GET /admin/users/paginated", auth.VerifyRoleMiddleware(admin.GetUsersPagines, "Administrateur"))
```

Test :

```
curl "http://localhost:8081/admin/users/paginated?page=2&limit=10" -H "Authorization: Bearer $TOKEN"
```

J.3 — Phrase à dire à l'oral

« J'ai paginé **côté client** car le volume est faible : je charge la liste une fois et j'affiche des tranches de 10 avec `slice`. Si la table devenait volumineuse, je passerais **côté serveur** avec une requête SQL `LIMIT / OFFSET` et un handler qui renvoie la page + le total, pour ne transférer que 10 lignes à la fois. »

I. Checklist « je viens de modifier, ça ne marche pas »

1. Erreur Go ? → `cd API && go build ./...` (lit l'erreur exacte).
2. Conteneur rebuild ? → `docker compose up -d --build backend|frontend`.
3. Cache navigateur ? → **Ctrl+Shift+R** (ou DevTools « Désactiver le cache »).
4. Bon code HTTP ? → F12 → Réseau (401/403/500/404/CORS).
5. La colonne SQL existe et l'ordre `SELECT` / `Scan` correspond ?